

Two ways to let an agent read on your behalf

— in plain terms

This note explains, without jargon, what the two architectures in this repo are doing and where they differ. It compares:

- **Ripple-slack-integration-architecture.png** (repo root) — the design Ripple drew, where the app writes its own code to fetch, encrypt, store, and refresh each user's access tokens in a DynamoDB table.
- **The AgentCore solution** (github.com/skopp002/ripple-slack-agentcore) — a working AgentCore agent that hands all of that token work to **AWS's managed "AgentCore Identity"** service instead of building it.

Both solve the same core problem. The difference is *who owns the hardest, most security-sensitive plumbing* — Ripple's own code, or AWS.

The problem both designs are solving

A user asks a question in Slack: *"What does the onboarding guide say about VPN access?"* The answer lives in company systems — GitHub, Google Drive, Confluence, Slack, Databricks.

The golden rule: **the agent must only show each person what that person is already allowed to see**. If Alice can't open a document, the agent must not read it for her either. The clean way to guarantee that is to have the agent call each system **as that user** — using *their* access, not a shared "service account" that can see everything.

To call a system "as Alice," you need Alice's **access token** — a short-lived digital key that proves "this request is on Alice's behalf." Getting that key, keeping it safe, and refreshing it when it expires is the entire ballgame. That is what these two designs handle differently.

The three "consent flows" (how a user's key gets obtained)

There are exactly three ways to obtain the key that lets the agent act as the user. The [AgentCore solution](#) labels them **M1**, **M2**, **M3**. Understanding these three is the key to reading both diagrams.

M1 — Consent flow ("ask the user once, then remember")

The user clicks "Approve" one time, per system, ever.

Think of the pop-up you've seen a hundred times: "*RippleBot wants to access your GitHub. Authorize?*" You click **Approve** once. From then on, the system quietly remembers your approval and the agent can read GitHub as you without asking again.

- **Human effort:** one click, the first time only.
- **Used for:** GitHub today; Slack and Confluence in the target state.
- **What has to be stored:** a long-lived "refresh token" (the thing that lets the agent keep getting fresh keys without bugging you again). **This is the sensitive secret that has to live somewhere safe** — and *where it lives* is the main difference between the two architectures.

M2 — Delegation flow ("the admin pre-approves for everyone; users never see a thing")

Nobody ever clicks anything.

A Google Workspace super-admin flips one switch, once, for the whole company ("domain-wide delegation"). After that, the agent can say to Google: "Act as alice@company.com" — and Google hands back a key scoped to **only** what Alice can see. Alice is never prompted, never sees a consent screen, and doesn't even know it happened.

- **Human effort:** zero for end users; one setup by an admin.
- **Used for:** Google Drive / Docs.
- **Why it's different from M1:** there's no per-user secret to store at all. The agent proves who the user is (from their verified login) and Google mints a fresh, narrowly scoped key on the spot every time. Nothing long-lived needs saving.

M3 — Token exchange flow ("log in once, reach everything — in theory")

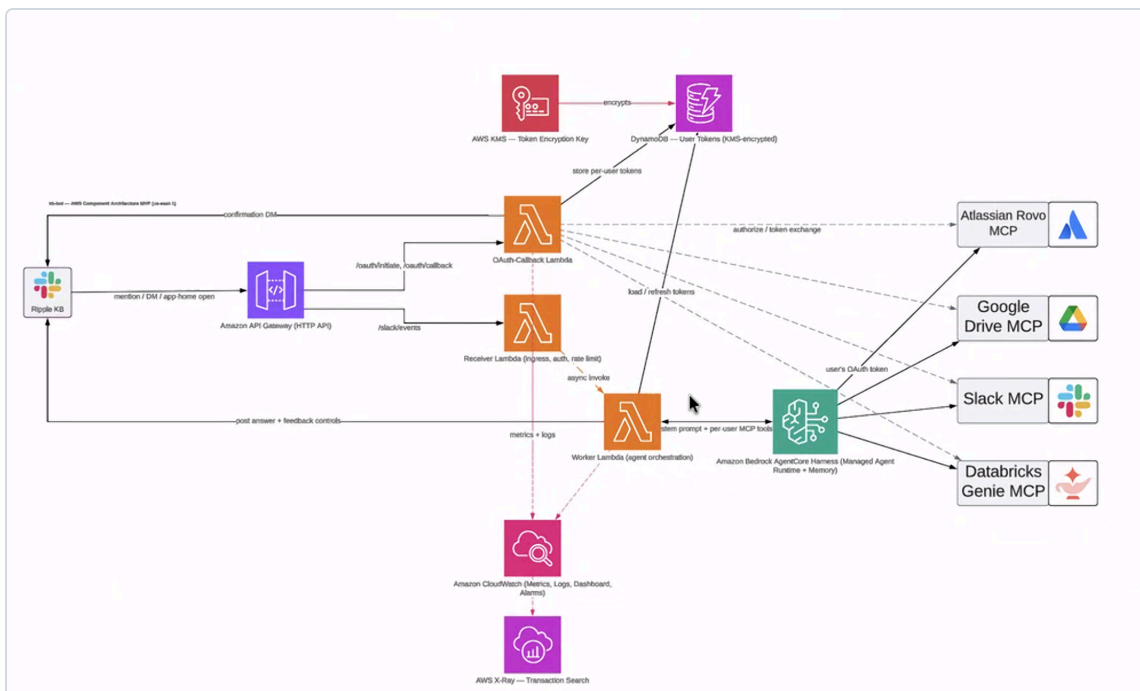
The dream: the user's single company login is automatically traded for a key to each downstream system, with no per-app consent and no stored secrets.

This is the "auth once, reach everything" idea (Okta calls it Cross-App Access / ID-JAG). The user's login token gets **exchanged** for a token each target system accepts.

- **Human effort:** zero — if it worked.
- **Status: drawn but deliberately switched off.** The [AgentCore solution](#) documents that M3 is blocked today by a three-part chain, and *all three* parts must work for any given system: (1) the company's identity provider has to be able to issue the special exchange token, (2) AWS AgentCore has to be able to ask for it (it currently can't — the API has no field for it), and (3) the target system (Slack, Confluence, etc.) has to *accept* it (most don't yet). Until all three line up, M3 stays as a planned path, not a live one.
- **Bottom line:** for Slack and Confluence, **M1 consent is the real plan** — not a temporary workaround — because their vendors' rules don't support the exchange.

One-line summary: M1 = user approves once and a secret is saved; M2 = admin pre-approves and no secret is ever saved; M3 = fully automatic but not yet possible.

What Ripple drew (Ripple-slack-integration-architecture.png)



Ripple's diagram is a **build-it-yourself token vault**. Reading it left to right:

1. A user interacts in Slack → **API Gateway** → **Receiver Lambda** (checks the request, rate-limits) → hands off to a **Worker Lambda** that runs the agent.
2. When a user needs to connect a system, an **OAuth-Callback Lambda** runs the "Approve" dance (the M1 consent flow) **in Ripple's own code**.
3. That Lambda then **writes the user's tokens into a DynamoDB table**, encrypted with a key from **AWS KMS**. (In the diagram: "store per-user tokens" and the KMS key that "encrypts" the table.)
4. On every later question, the Worker Lambda **reads the token back out of DynamoDB and refreshes it if it's expired** (the "load / refresh tokens" arrow), then passes the user's token to the **Bedrock AgentCore Harness**, which calls the MCP servers (Atlassian, Google Drive, Slack, Databricks Genie).

In short: **Ripple's application code owns the entire token lifecycle** — obtaining tokens, encrypting them, storing them in a database Ripple runs, reading them back, refreshing them, and doing any token exchange. AgentCore in this picture is just the thing that runs the agent and gets handed a token.

What that means in practice — Ripple would own, forever:

- A **DynamoDB table full of every user's live access keys** — a high-value target that must be encrypted, access-controlled, backed up, and audited.
- The **KMS encryption key** and its rotation.
- The **refresh logic** — code that must correctly renew tokens before they expire, handle revocation, and never leak or log a secret.
- The **OAuth handshake code** for every provider, and the security review that comes with holding client secrets.

None of that is wrong — it's a standard, well-understood pattern. It's just a lot of security-critical code and infrastructure that Ripple has to build, secure, and maintain.

(Full code: github.com/skopp002/ripple-slack-agentcore)

The diagram's shorthand, in plain terms:

What happens on a single question – delegation (Google Drive, no consent):

1. The user asks a question in plain language — no source is named, no permission requested.
2. The already-issued login token rides along as proof of who is asking.
3. Inside the agent, that token is re-verified from scratch (signature, issuer, audience, `email_verified` must be true, and the address must be in the allowed domain) before the agent will act as anyone. This re-check is the whole safety argument for impersonation.
4. The agent asks Google to "act as this verified email"; Google returns a key scoped to just that one user. **No consent screen** — the admin pre-approved once, domain-wide.
5. Drive returns only documents that user can already open (Google trims it, not the agent), and the answer comes back with citations and a HIGH/MEDIUM/LOW confidence band.

What happens on a single question — consent (GitHub, one click): steps 1–3 are identical. Then, the *first* time this user needs GitHub, AgentCore Identity has no token for them, so it hands back a consent URL; the user clicks **Approve** once; the resulting token is vaulted; and from then on every question is silent. GitHub then returns only what that user's own grants allow.

The one human action anywhere in the whole system is that single "Approve" click in the consent flow. The delegation flow asks the user for nothing, ever.

Same goal, same permission guarantee — but it **deletes the DynamoDB table, the KMS key, and the refresh code** and hands that job to a managed AWS service: **AgentCore Identity**.

- The user logs in once through the company identity provider and gets a signed login token (a JWT). AgentCore's front door validates it, and the agent re-checks the signature itself before it will act as anyone.
- **Consent flow (M1, GitHub):** when a user first needs GitHub, they get the one "Approve" click. The resulting token is **vaulted inside AgentCore Identity** — AWS stores, encrypts, and refreshes it. Ripple never sees or holds it. There is **no DynamoDB table and no KMS key to run**.
- **Delegation flow (M2, Google Drive):** no consent screen at all. The agent passes the user's verified email to Google, which returns a key scoped to just that user. Nothing is stored.
- Either way, **the source system itself does the permission-trimming** — the agent calls it as the user, so Alice only ever gets Alice's results.

The heart of the difference, in one sentence: Ripple's diagram has the app *manage access tokens in its own DynamoDB table*; the solution has *AgentCore Identity manage the tokens*, so that database — and all the code around it — simply doesn't exist.

Side-by-side

	Ripple's diagram (build-your-own)	The AgentCore solution (managed)
Where user tokens live	DynamoDB table Ripple runs , KMS-encrypted	AgentCore Identity vault (AWS-managed)
Who runs the "Approve" dance (M1)	Ripple's OAuth-Callback Lambda	AgentCore Identity
Who refreshes expiring tokens	Ripple's Worker Lambda code	AgentCore Identity
Encryption key to manage	Yes — a KMS key Ripple owns	No — handled inside the service
No-consent path for Google (M2)	Not shown	Yes — delegation, zero user clicks
Token exchange (M3)	Implied via the Lambda	Documented, wired, but off (not yet possible)
Security surface Ripple owns	Token DB + KMS + refresh + OAuth code	Just: verify the login, then call the source
Permission trimming	The source system	The source system (same guarantee)

A potential path: Okta Cross-App Access (XAA / ID-JAG)

There is a third mechanism on the horizon — **M3, token exchange** — that promises "log in once, reach everything" with **no consent screen and no stored token at all**. Okta's version is called Cross-App Access (XAA). An [AWS sample](#) shows it working today: the user's single Okta login is exchanged, per request, for a short-lived token the target accepts, so the user is never asked to approve anything.

Worth being clear-eyed about what it does and does not buy:

- **It works only where the target speaks ID-JAG.** The exchange only succeeds if the source's own authorization server accepts an Okta-issued ID-JAG token. That realistically means a source **guarded by an Okta-fronted MCP server** — which today is **potentially Atlassian (Confluence) only**. Google Drive can't use it (Google requires its own service-account-signed assertion — hence M2 delegation), and Slack can't (its server accepts the plain consent grant only — hence M1 consent).
- **If not every source supports ID-JAG, it becomes an unnecessary loophole.** Because it removes the consent step for XAA-capable sources *only*, adopting it does not give you one clean model — it gives you a **fourth** code path alongside M1 and M2, benefiting a single connector. You still run consent for Slack and delegation for Google; XAA just adds a special case for the one Okta-guarded target. That extra path carries its own cost: in the AWS sample the exchange is driven by the **agent's own code**

holding an Okta workload private key — not by the managed AgentCore Identity vault — which puts a credential and token-validation logic back into the code this solution otherwise keeps out.

- **What it does *not* reintroduce:** no DynamoDB token table and no KMS key come back — the exchanged tokens are short-lived and per-request, so nothing is stored. That part of the win holds.

Bottom line on XAA: treat it as a *potential optimisation for Okta-guarded sources* (potentially Atlassian only), not as a replacement for M1/M2. Until a meaningful share of sources support ID-JAG on their own authorization servers, the consent (M1) and delegation (M2) mechanisms above remain the plan of record — and XAA would be a narrow special case, not the single "auth once, reach everything" flow it appears to promise.

The takeaway

Both designs reach the same promise: *the agent only ever sees what the signed-in user is allowed to see, because it calls each system as that user.*

They differ in **who carries the token**. Ripple's diagram carries it themselves — building and forever securing a database of everyone's access keys plus the code to encrypt, refresh, and exchange them. The [AgentCore solution](#) hands that entire burden to **AgentCore Identity**, so the most dangerous, highest-maintenance part of the system — the vault of live user credentials — is something AWS runs, not something Ripple has to protect.

Fewer moving parts Ripple has to secure, the same access guarantees for users, and a clear, documented path (M1 consent, M2 delegation, M3 exchange when it's ready) for adding each new knowledge source.

— Sunita Koppar